

Validación de los 4 Requerimientos Mínimos

Dance Labo · F-04 · Sprint 6

Última actualización: Mayo 2026

Sistema: Dance Labo — LSTM sobre huesos Mixamo (X Bot / El Bueno)

Checklist General

#	Requerimiento	Estado
1	Detecta réplica y muestra feedback visual	COMPLETO — LiveEvaluator + FeedbackUI.cs
2	Robot 3D mejorado (rig + aprende movimientos)	COMPLETO — LSTM entrenado con Secuencia1 y sixseven
3	2+ secuencias grabadas y modelo entrenado	COMPLETO — 32 archivos, 2 movimientos
4	Mapeo de huesos documentado	COMPLETO

1 Detecta Réplica y Muestra Feedback Visual

El sistema detecta en tiempo real cuándo el estudiante replica el movimiento del robot y muestra colores (verde / amarillo / rojo) por segmento corporal.

Pipeline completo implementado

```
Webcam
├─ YOLO (detección persona)
├─ MediaPipe (33 landmarks 3D)
├─ MixamoMapper (9 vectores de huesos normalizados)
│   ├── UDP → MixamoAnimator.cs (mueve el robot en Unity)
│   └─ LiveEvaluator.evaluate(bones)
│       ├── ScoringEngine.evaluate_frame()
│       │   ├── Score por segmento (0.0 – 1.0)
│       │   ├── Overlay en video (cv2.putText)
│       │   └─ QLabel en UI Python
│       └─ UDP → MovementComparator.cs
│           └─ FeedbackUI.cs (Unity)
```

Criterios de aceptación — Req 1

- [x] Detección de réplica con feedback verde (score $\geq 85\%$)
- [x] Colores por segmento: Verde $\geq 85\%$, Amarillo 60-84%, Rojo $< 60\%$
- [x] Score general visible en la UI de Python (lbl_live_score)
- [x] Score visible en Unity via MovementComparator.cs + FeedbackUI.cs
- [x] Transición suave con lerp (sin parpadeo) — LerpSpeed configurable
- [x] Overlay de score en el frame de video (cv2 semitransparente)
- [x] Evento OnReplicaDetected cuando score $> 85\%$ por $> 1s$
- [x] Evento OnGoodStreak cuando score verde por $> 3s$ consecutivos
- [x] Barra de progreso de la secuencia de referencia

Cómo activar la evaluación en vivo

- # 1. En Unity: Play Mode con "El bueno" en escena
- # 2. En Python:
`cd motion_ml_app`
`python main.py`
- # 3. En la UI:
 - # - Escribe el nombre del movimiento (ej: "Secuencia1")
 - # - Pulsa "Usar Cámara"
 - # - Pulsa "Evaluar en Vivo"
 - # → El score se muestra en pantalla y se envía a Unity en tiempo real

2 Robot 3D Mejorado

El modelo LSTM aprende y reproduce los movimientos grabados con el robot. En mayo 2026 se integró el nuevo rig LABO new rig (Blender → Unity Humanoid) con soporte completo de caderas, columna vertebral, cuello y piernas.

Mejoras del rig (P3 — Mayo 2026)

Mejora	Detalle
Nuevo avatar	LABO new rig.fbx / .blend con jerarquía DEF-*
Swap L/R	swapLeftRight corrige rigs en espejo de Blender
Correcciones de eje por grupo	Leg / Arm / Torso con flags negateX/Y/Z independientes
Amortiguación Z	legZMultiplier, armZMultiplier, torsoZMultiplier (MediaPipe Z es ruidoso)
Cuello	Vector nariz → promedio hombros con neckZMultiplier=0
Diagnóstico	logDiagnosticsOnStart=true imprime estado de todos los huesos en consola

Arquitectura del modelo (E-01)

Parámetro	Valor
Tipo	LSTM Generativo (MotionLSTMGenerator)
Input	ID de movimiento (embedding)
Output	27 valores por frame (9 huesos × xyz)
Hidden size	256
Capas LSTM	3
Epochs	150
Optimizador	Adam + StepLR (÷2 cada 50 epochs)
Criterio	MSELoss

Proceso de generación de referencia

```
predictor = MotionPredictor("data/motion_model.pt")
frames = predictor.predict_sequence("Secuencia1")
# → lista de dicts [{bone: {x,y,z}}, ...] para cada frame
```

Criterios de aceptación — Req 2

- [x] LSTM entrenado y guardado (data/motion_model.pt)
- [x] Genera secuencias de referencia por movimiento en < 5s de carga
- [x] Label map guardado y cargable (data/label_map.pt)
- [x] Script reproducible: mismos datos = mismo modelo (train.py)
- [x] El modelo se re-entrena automáticamente con python train.py cuando hay nuevas secuencias
- [x] Nuevo rig integrado con soporte de piernas, torso y cuello (P3)

3 Secuencias Grabadas y Modelo Entrenado

El modelo fue entrenado con el dataset depurado de 32 secuencias de 2 movimientos distintos utilizando el LABO new rig (mayo 2026).

Secuencias disponibles (data/sequences/)

Movimiento	Archivos	Total frames aprox.	Descripción / Fuente
Secuencia1	26 archivos (0 a 25)	~8,400	Secuencia de movimiento principal de baile con el rig nuevo (LABO new rig).
sixseven	6 archivos (26 a 31)	~1,000	Secuencia de movimiento secundaria / control.

Total: 32 archivos JSON, ~9,400+ frames de datos reales.

Resultado del entrenamiento más reciente

```
=====
Dance Labo - Entrenamiento LSTM
=====

Movimientos encontrados: {'Secuencia1': 0, 'sixseven': 1}
Total de muestras:      32
Longitud máx. secuencia: 153 frames

Dispositivo: cpu
Clases: 2 | Epochs: 150 | LR: 0.001

Epoch  1/150 | Loss: 0.162211 | Best: 0.162211 | #####
Epoch 10/150 | Loss: 0.098318 | Best: 0.098114 | #####
Epoch 20/150 | Loss: 0.087878 | Best: 0.087878 | #####
Epoch 30/150 | Loss: 0.067106 | Best: 0.067106 | #####
Epoch 40/150 | Loss: 0.060582 | Best: 0.060582 | #####
Epoch 50/150 | Loss: 0.053173 | Best: 0.052081 | #####
Epoch 60/150 | Loss: 0.056307 | Best: 0.050861 | #####
Epoch 70/150 | Loss: 0.051994 | Best: 0.049761 | #####
Epoch 80/150 | Loss: 0.050571 | Best: 0.048956 | #####
Epoch 90/150 | Loss: 0.048695 | Best: 0.048282 | #####
Epoch 100/150 | Loss: 0.049831 | Best: 0.048126 | #####
Epoch 110/150 | Loss: 0.046836 | Best: 0.046077 | #####
Epoch 120/150 | Loss: 0.048370 | Best: 0.046061 | #####
Epoch 130/150 | Loss: 0.047091 | Best: 0.046061 | #####
Epoch 140/150 | Loss: 0.048465 | Best: 0.046061 | #####
Epoch 150/150 | Loss: 0.047929 | Best: 0.046061 | #####

Entrenamiento completo.
Modelo guardado en: data/motion_model.pt
Label map en:      data/label_map.pt
Loss final:        0.046061
```

Cómo re-entrenar

```
cd motion_ml_app
python train.py
# Al terminar, el nuevo modelo reemplaza data/motion_model.pt automáticamente
```

Criterios de aceptación — Req 3

- [x] 2+ secuencias JSON en data/sequences/ (tenemos 32)
- [x] Modelo distingue los movimientos conocidos
- [x] Score continuo 0.0–1.0 por frame (similitud coseno)
- [x] Modelo guardado en .pt (PyTorch portable)
- [x] Script de entrenamiento reproducible (train.py)
- [x] Validación disponible (inference.py, benchmark.py)

4 Mapeo de Huesos Documentado

El sistema usa 9 huesos Mixamo para el modelo ML (scoring), y envía adicionalmente Hips, Spine1 y Neck a Unity para animar el cuerpo completo.

Feature vector ML por frame (27 valores — scoring)

Índice (x3)	Hueso Mixamo	Segmento	Peso en score
0–2	mixamorig:LeftArm	Brazo izquierdo	25%
3–5	mixamorig:RightArm	Brazo derecho	25%
6–8	mixamorig:LeftForeArm	Brazo izquierdo	(incluido)
9–11	mixamorig:RightForeArm	Brazo derecho	(incluido)
12–14	mixamorig:LeftUpLeg	Pierna izquierda	20%
15–17	mixamorig:RightUpLeg	Pierna derecha	20%
18–20	mixamorig:LeftLeg	Pierna izquierda	(incluido)
21–23	mixamorig:RightLeg	Pierna derecha	(incluido)
24–26	mixamorig:Spine	Torso	10%

Huesos adicionales enviados a Unity (animación completa — no ML)

Hueso Unity	Cálculo Python	Nota
mixamorig:Hips	Vector caderas → hombros	Centro de masa
mixamorig:Spine1	Igual que Spine	Puede refinarse con más datos
mixamorig:Neck	Vector hombros → nariz (LM 0)	Z suprimido (neckZMultiplier=0)

Correspondencia MediaPipe → Hueso Mixamo

Landmark MediaPipe	ID	Hueso Unity
nose	0	destino estimado de mixamorig:Neck

Landmark MediaPipe	ID	Hueso Unity
left_shoulder	11	origen de mixamorig:LeftArm
left_elbow	13	destino de mixamorig:LeftArm / origen de LeftForeArm
left_wrist	15	destino de mixamorig:LeftForeArm
right_shoulder	12	origen de mixamorig:RightArm
right_elbow	14	destino de mixamorig:RightArm / origen de RightForeArm
right_wrist	16	destino de mixamorig:RightForeArm
left_hip	23	origen de mixamorig:LeftUpLeg + Hips/Spine
left_knee	25	destino de mixamorig:LeftUpLeg / origen de LeftLeg
left_ankle	27	destino de mixamorig:LeftLeg
right_hip	24	origen de mixamorig:RightUpLeg + Hips/Spine
right_knee	26	destino de mixamorig:RightUpLeg / origen de RightLeg
right_ankle	28	destino de mixamorig:RightLeg
Promedio hombros 11+12	—	origen de mixamorig:Spine / Spine1 / Hips
Promedio caderas 23+24	—	destino de mixamorig:Spine / Hips

Sistema de coordenadas

MediaPipe: +X derecha, +Y abajo, +Z atrás (lejos de cámara)

Unity: +X derecha, +Y arriba, +Z adelante (hacia la cámara)

Conversión Python: [x, -y, -z] (MixamoMapper._mp_to_unity_coords)

Reglas de visibilidad y corrección de ejes

```
# MixamoMapper — pose_extraction/mixamo_mapper.py
visibility_threshold = 0.3 # freeze de última pose válida si baja
neck_z_multiplier = 0.0 # suprime Z ruidoso del cuello
```

```
# MixamoAnimator.cs — Unity (configurable por grupo desde Inspector)
# Brazos: armZMultiplier, enableArmZClamp, armZClampMin
# Piernas: legZMultiplier, legNegateX/Y/Z
# Torso: torsoZMultiplier, torsoNegateX/Y/Z
# Cuello: neckZMultiplier
```

Criterios de aceptación — Req 4

- [x] 9 huesos mapeados con índice fijo y orden documentado
- [x] 3 huesos adicionales (Hips, Spine1, Neck) para animación completa
- [x] Función bones_dict_to_vector() convierte huesos a vector 27D
- [x] Segmentos corporales con pesos que suman 1.0
- [x] Huesos ausentes se llenan con cero (sin crash)
- [x] Segmentos sin datos se excluyen del score (normalización automática)
- [x] Freeze de última pose válida cuando visibility < umbral
- [x] Restricción de eje Z para brazos (configurable)
- [x] Correcciones de eje por grupo en Unity (legNegateX/Y/Z, armZMultiplier, etc.)

Archivos Clave del Sistema

```
motion_ml_app/
├── main.py          # Punto de entrada — lanza la UI
├── train.py         # Entrena el LSTM con data/sequences/
├── inference.py     # Valida el modelo con datos reales
├── benchmark.py     # Mide latencia E-05 (<100ms)
├── check_env.py     # A-01: verifica dependencias
├── ui/
│   └── main_window.py # UI PyQt6 completa
├── ml/
│   ├── live_evaluator.py # E-04: evaluación frame a frame
│   ├── scoring_engine.py # D-06: motor de puntuación
│   ├── predictor.py     # Generación de secuencias LSTM
│   ├── trainer.py       # Lógica de entrenamiento
│   ├── dataset_builder.py # Carga y normaliza JSONs
│   └── model.py         # MotionLSTMGenerator
├── pose_extraction/
│   └── mixamo_mapper.py # MediaPipe → vectores Mixamo
├── communication/
│   ├── udp_server.py   # Envía JSON por UDP a Unity
│   ├── MovementComparator.cs # D-01/D-02: Unity recibe scores
│   └── FeedbackUI.cs   # D-05: colores por segmento en Unity
Assets/ (Unity)
├── MixamoAnimator.cs   # Anima el rig
├── MovementComparator.cs # D-01/D-02
├── FeedbackUI.cs       # D-05
├── UDPClient.cs        # Recibe JSON de Python
├── LABO new rig.fbx    # Nuevo avatar (P3 — mayo 2026)
└── Editor/RigMapper.cs # Herramienta de mapeo de huesos
data/
├── motion_model.pt     # Modelo LSTM entrenado
├── label_map.pt        # {nombre: id}
├── historial_scores.json # Historial de intentos
├── benchmark_report.json # Resultado del benchmark E-05
└── sequences/         # 32 JSONs (Secuencia1 y sixseven)
```

Cómo Hacer la Demo Final

1. Preparación

```
# En la PC de demo:
cd motion_ml_app
pip install -r requirements.txt # solo primera vez
```

2. Unity

- Abrir la escena con 'El bueno'.
- Asegurarse de que `MixamoAnimator.cs` y `MovementComparator.cs` están en el mismo `GameObject`.
- Asegurarse de que `FeedbackUI.cs` está configurado con sus referencias.
- Presionar Play.

3. Python

```
python main.py
```

4. En la UI

1. Pulsa **Usar Cámara** (selecciona la cámara correcta con el combo si hay varias).
2. Escribe el nombre del movimiento (ej: `Secuencia1`).
3. Pulsa **Evaluar en Vivo**.
4. El robot en Unity se mueve con tu pose, y el score aparece en pantalla.

5. Plan B (si falla la cámara)

- Cargar un video pregrabado con **Cargar Video**.
- El pipeline funciona igual con video pregrabado.